第088讲 实验手册

运行准备：每一步都有检查点

建议Python 3.12，CPU即可。先在终端进入本讲目录，创建独立虚拟环境，再按README安装依赖。运行python experiment.py会生成outputs/results.json和figures中的图；运行python -m unittest -v test_experiment.py会执行数值测试。首次安装PyTorch可能需要下载较大CPU轮子，训练本身不需要联网。

请先确认python -c "import torch; print(torch.**version**)"成功，再运行实验。若提示找不到模块，通常是安装包的Python与运行脚本的Python不是同一个，不要反复混用pip和pip3。使用python -m pip能明确安装目标。

实验默认CPU单线程、固定随机种子。结果数值可随版本产生细微变化，耗时会随机器负载明显变化。验收依据是数学容差与机制，不要求你的毫秒数和本包完全一致。不要删除失败对照，它们是本讲证据的一部分。

已执行experiment.ipynb含实际IPython执行输出和图像，可先阅读再重跑。Notebook与脚本使用同一实验函数，避免维护两套相互漂移的实现。运行会覆盖本讲outputs和figures中生成的同名文件，请先备份自己修改后的结果。

实验A：在改代码前先写预算

打开experiment.py的data、model、train三个函数。填一张实验记录：训练384、验证128、特征6、隐藏宽32、更新120、batch64、每次运行曝光7680、种子11/23/37。确认三种优化配置采用相同数据与索引生成规则。注意SGD与AdamW的学习率不同，这里比较配置而非只比较更新公式。

运行后读取results.json的runs。对每个配置计算最终验证loss均值和样本标准差，保留三个原始值。再打开01_budget_schedule.png，回答哪条线早期下降快、哪条后期较低。若结论依赖单个种子，写成不稳定观察，不要用平均线掩盖。

实验B：亲手制造累积缩放错误

读accumulation函数：一次大batch含37条，微批16、16、5。正确分支乘每段样本数/37，错误分支没有缩放。比较correct_max_abs_error与unscaled_max_abs_error。正确差应小于1e-6，错误差应显著非零。

扩展：把正确分支改成每段除以3，另存结果，不要覆盖交付基线。预测短微批被过度赋权，再测实际误差。然后改成三段等长，验证除以3何时成立。解释你检验的是同一参数点的梯度，而不是训练多步后的准确率。

实验C：让中断恢复成为可验证命题

第57步写checkpoint.pt，从磁盘读取后继续到120步。检查resume.exact_max_abs_error接近0；weights_only_max_abs_error明显大于前者。打开checkpoint看看键名：model、optimizer、step、rng、history。

不要加载来源不明的checkpoint。本例只加载刚由自己生成的文件，且使用weights_only=True。若加入NumPy标量到checkpoint，安全加载可能拒绝该类型；优先转换成Python原生数值，不要为省事关闭安全选项。

进一步实验：在模型里加入dropout，完整保存并恢复CPU随机状态；比较漏掉rng的失败分支。若只在更新边界保存，本例无需保存尚未使用的梯度；如果你改成微批中途保存，必须重新定义状态集合。

提交与验收

提交三张图、results.json、测试日志和一页报告。报告必须说明预算口径、验证集用途、恢复时钟、累积权重以及实验范围。不得把三个种子的验证均值称为独立测试性能。通过标准：完整恢复与大batch累积两项数值核验通过，失败对照确实失败，而且能解释为什么。